MV QUIZZER PLUGIN SYSTEM DOCUMENTATION

REQUIRED FILES

Javascript

- getQuestion.js
- questionDatabase.js

PNG Images

- MVQ_audio.png
- MVQ_correctAnswer.png
- MVQ_wrongAnswer.png
- MVQ_picBG.png

OGG Audio

- MVQ_correctAnswer.ogg
- MVQ_wrongAnswer.ogg

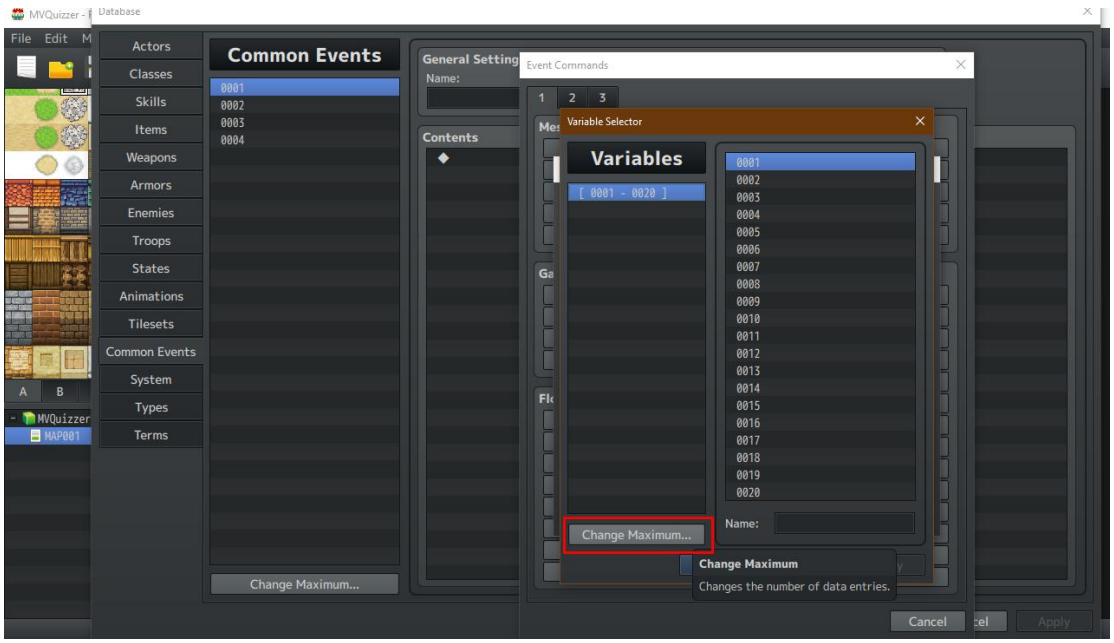
PLUGIN SETUP STEPS

1. Place getQuestion.js and questionDatabase.js in the **/js/plugins** folder of your game. Then add getQuestion.js and questionDatabase.js to your plugin list and make sure that they are set to ON.

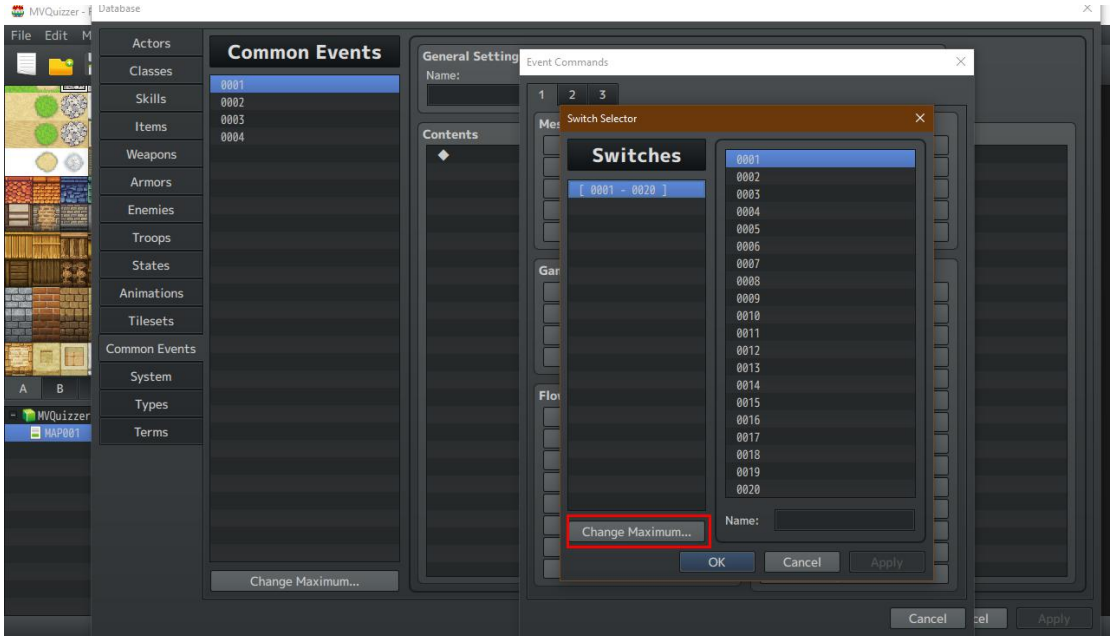
2. Place MVQ_audio.png, MVQ_correctAnswer.png and MVQ_wrongAnswer.png image files in the **/img/pictures** folder for your game.

3. Place MVQ_correctAnswer.ogg and MVQ_wrongAnswer.ogg ins the **/audio/se** folder.

4. Change maximum number of **Variables** in your game to 1000. The plugin uses variables 991 to 999. This allows the plugin to be added to a game that already has a large number of variables in use without making any changes to existing variables.



5. Change the maximum number of **Switches** in the game to 1000. Switches 991 and 992 are currently used. 991 is switched on when a correct answer is registered. 992 is switched on when a wrong answer is registered. Switches 993 to 999 are reserved for future feature implementation.



6. Add questions to questionDatabase.js in a text editor. The plugin will proceed through the questions in the order that they're placed in that file. After the last question, the plugin will automatically cycle back to the first question in the database.

That's it! Now let's look at how to create questions and add them to your database, along with what the plugin can do with questions that are in the database. Then in the last section we'll look at how the plugin interacts with RPG Maker MV and how you can get the most out of the plugin.

You will need a text editor for creating your database. Any text editor will work, but Notepad++ is highly recommended for its many helpful features.

DATABASE CREATION

1. Create new file in text editor and name it questionDatabase.js (or open up a blank questionDatabase.js template file).

2. Type the following into the text editor. It must be typed exactly like this:

```
var questionDatabase = {"Questions" : [

]}
```

This is the core of the Question Database, which is all contained in the questionDatabase.js file. All of the questions in the database will go in between the two brackets [ ]. Examples will follow.

3. A Question Database with one question in it will look like the following:

```
{
"Note":"Any important notes on the question go here",
"GUID":"MVQ-1234-abcd-5678",
"E":0,
"Q_T":4,
"Q":"Have you ever seen a multiple choice question with four choices?",
"T":,
"I":0,
"A":0,
"C_A":"Yes",
"A2":"No",
"A3":"Maybe",
"A4":"What was the question again?",
"C_S":0,
"S":"",
"R_T":"Gold",
"R_I":0,
"R_A":100,
"P_T":"HP",
"P_I":0,
"P_A":50,
"O_L":0,
},
```

It's important to note that the line spacing is not needed in the database and is not included in the Question Builder associated with this plugin (more on the Question Builder later). Line spacing is included here for human readability. In a moment we'll go over in detail what information is included in each question.

The plugin will read the information in this Question and will present it in the game when the plugin is called. Once the player responds, the plugin will evaluate the answer automatically.

4. Save the questionDatabase.js file and it will now be updated with the questions that you have added to it.

WHAT'S IN A QUESTION?

Note - Used only for human readability. Place notes about the question here, e.g. a description of the associated image or audio file.

GUID - Unique identifier for question. This must match the audio file name and the image name for the related question. In other words, the GUID serves as the Question ID that links the question to any associated media. The Question Builder creates pseudo GUIDS (not guaranteed to be unique, but 99.99999%+ likely to be so. All Question Builder GUIDs begin with MVQ_ in order to aid with image and audio file organization in your game folders. You can also manually create GUIDs if you would prefer to have something more human readable.

E - Is the question encoded? 1 indicates the question and answers are encoded. 0 indicates that they are NOT encoded. Encoding relies upon the Question Builder and the encoding method is hardcoded in the plugin. This just means that the work is already done for you and there's no need to worry about creating an encoding method. If you know what you're doing and want access to the encoding/decoding functions, just send an email and we'll send you the unobscured functions. The

functions are obscured merely to provide a small layer of protection against cheating on the questions.

Q_T - Question Type.

1 = Short Answer,
2 = 2 answer Multiple Choice
3 = 3 answer Multiple Choice
4 = 4 answer Multiple Choice
5 = 5 answer Multiple Choice
6 = RESERVED
9 = True or False

Q - Question text

T - Time limit for question (in ticks at 60 ticks per second T: 900 = 15 second time limit)

I - Image - Does the question have an associated image file. 0 = no, 1 = yes. If yes, the image file is pulled using the GUID as the file name. [GUID].png (DO NOT ADD .PNG file ending)

A - Audio - Does the question have an associated audio file. 0 = no, 1 = yes. If yes, the audio file is pulled using the GUID as the file name. [GUID].wav (DO NOT ADD .WAV file ending)

C_A - Correct Answer - This is the correct answer to the question.

A2, A3, A4, A5 - Incorrect answers to the question. These are mixed up in the program to deliver the potential answers in random order.

C_S - Case Sensitive - Only used for Short Answer questions. Is this question case sensitive? 0 = no, 1 = yes. This is used if proper capitalization is important.

S - Standard - If there is an associated standard (industrial, educational, etc) then this is where to note that information so that it can be recorded in question result output.

R_T - Reward Type - What type of reward does the correct answer give to the player (Gold, Item, Weapon, Armor, HP, MP, XP, Skill, State Add, State Remove, Start Battle).

R_I - Reward Id - What is the Id of the Skill, Item, Armor, Weapon, State, Enemy Troop, etc of the correct answer reward.

R_A - Reward Amount - How much of R_T / R_I is given to the player upon correct answer.

P_T - Penalty Type - What type of penalty does the wrong answer give to the player (Gold, Item, Weapon, Armor, HP, MP, XP, Skill, State Add, State Remove, Start Battle).

P_I - What is the Id of the Skill, Item, Armor, Weapon, State, Enemy, etc of the correct answer penalty.

P_A - Penalty Amount - How much of P_T / P_I is given to the player upon correct answer.

O_L - Only Leader - If O_L = 1 then only the party leader will receive the award or penalty on correct or wrong answers. If O_L = 0 then the entire party receives the reward or penalty.

Now that you know what is information is included in each Question in the Question Database, let's take a look at what you can do with a Question.

- You can create Multiple Choice questions with two to five (2 - 5) answers. Answers are presented as choices in a randomly shuffled order.
- You can create True / False questions.
- You can create Short Answer questions.

All questions can be presented with image files. The image files are connected to the question's GUID. The GUID must match the file name of the associated image. **Image files must be in .png format.** Images are shown using RPG Maker MV's PICTURE system. Picture 98 and 99 are used. DO NOT USE THESE PICTURES IN YOUR GAME. They are reserved for this plugin.

Short Answer graphics are automatically resized to **22px0 x 220px**. Multiple Choice and True/False questions can have graphics up to **480px x 480px** (there is a little flexibility here, but that's a good rule of thumb).

Multiple Choice and True / False questions can also be presented with audio files. The audio files are connected to the question's GUID and again the GUID must match the associated audio file. **Audio files must be in .ogg format.** Audio support for Short Answer questions will be available in future versions of this plugin.

As soon as the question is presented to the player, a timer is automatically started. This times the player's response time. Results are rounded to the nearest tenth (0.1) of a second. **Timer results are passed to Game Variable 995**. You can use common event listeners or other eventing methods to interact with the timer result for each question.

Correct Answers trigger **Switch 991** to ON.
Wrong answers trigger **Switch 992** to ON.
Both of these switches are automatically reset to OFF when the next question is called. Again, common event listeners or other eventing methods can be used to interact with these switches.

Game Variable 996 stores the current Correct Answer Streak. This streak counter works by adding one (1) every time a correct answer is given. It resets to zero (0) upon registering a wrong answer. **Game Variable 997** stores the current Wrong Answer Slump. This streak counter works the same way and resets to zero (0) upon registering a correct answer. Common event listeners or other eventing methods can be used to interact with these counters.

USING THE QUESTION BUILDER

1. Open up questionDatabse.js in a text editor. It should look like this when you begin:

```
var questionDatabase = {"Questions" : [

    ]}
```

2. Open QuestionBuilder.html in a browser window. You do not need to be online to do this. You are simply opening a local html file in the browser. Everything is done on the client side (your computer).

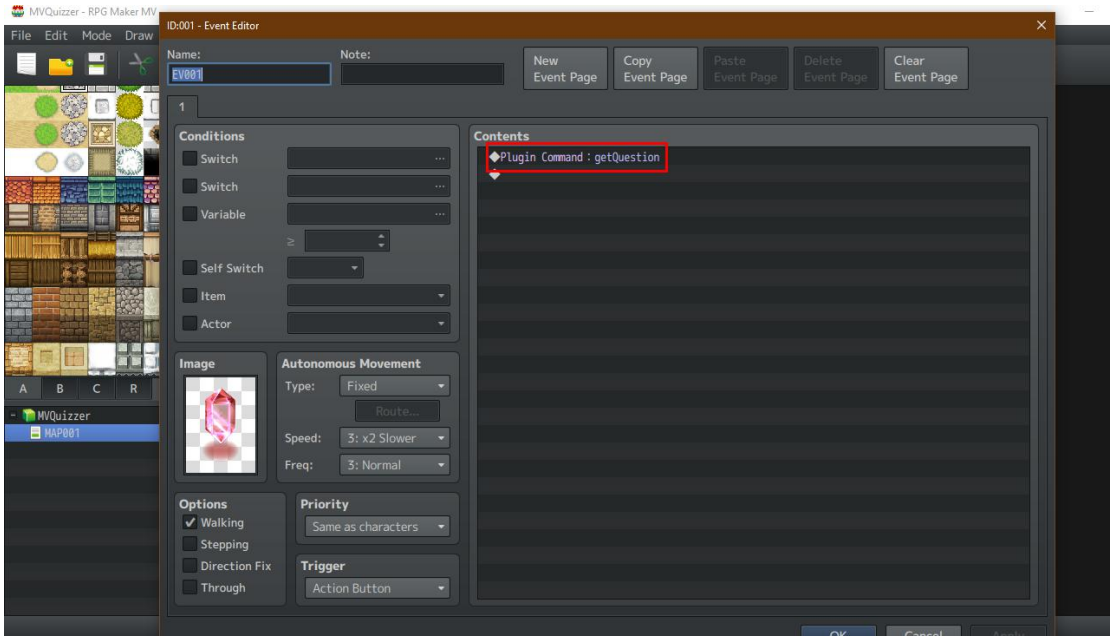
3. Select the kind of question you want to create then fill out the form. Once you click the **CREATE** button you will see a string of text in your browser window. Copy the full string of text and then paste it into your database. I should look like this:

```
 var questionDatabase = {"Questions" : [
```

{"Note":"just a note","GUID":"MVQ-609c-728c-467d","E":0,"Q_T":1,"Q":"Can you see the short answer question here?","I":0,"C_A":"Yes,"C_S":0,"S":"","R_T":"Item","R_I":3,"R_A":5,"P_T":"HP","P_I":0,"P_A":100,"O_L":0},
]}

It looks a little funny because it's formatted without any line breaks. In your text editor it may appear as a single long line of text. That's fine. Keep playing around with the question builder and adding questions until you have the hang of it. It's fairly simple.

Finally, we are ready to create an event in the game that asks a question.



That's it! The plugin will pull the question from questionDatabase.js once it is triggered by this event. The game will automatically save using the last loaded save slot. This is to discourage cheating on the questions. Questions are pulled in the order they appear in the Question Database.

Check out the demo game to see how the plugin works in practice. Whether you're looking to add an interesting level of player interaction with game trivia or to create an educational game, this plugin will enormously reduce the workload necessary to.

INFORMATION ON VARIABLES AND SWITCHES

Game VARIABLES and SWITCHES begin at 991 and are reserved up to 999 for this plugin.
Game Variable 991 - Stores text input strings for answering Short Answer questions
Game Variable 992 - Stores the question number and determines which question to pull next from the database.
Game Variable 993 - Stores the total number of correct answers
Game Variable 994 - Stores the total number of wrong answers
Game Variable 995 - Stores Question Timer Result
Game Variable 996 - Stores Streak Count
Game Variable 997 - Stores Slump Count
Game Variable 998 - Stores Reward ID
Game Variable 999 - Stores Reward Amount

Game Switch 991 - Switch that activates upon correct answer response
Game Switch 992 - Switch that activates upon wrong answer response
Game Switches 993 - 999 - RESERVED

PICTURES 98 and 99 USED. DO NOT USE THESE PICTURE NUMBERS IN YOUR GAME

CREDITS

CmdInp.js - Darkkitten
The Short Answer system relies heavily on Darkkitten's CmdInp.js

PopupWindow.js - Vlue of Daimonious Tails
The Reward / Penalty system uses PopupWindow.js to display information to the player

Stop Sound Effect function - Chaucer